

Горизонтальное секционирование таблиц

При разработке корпоративных приложений может возникнуть проблема уменьшения производительности, вызванная значительным увеличением объема отдельных таблиц с большой OLTP-нагрузкой.

Основные причины уменьшения производительности, вызванные большими объемами таблицы:

1. Autovacuum требуется больше времени на поиск и очистку "мертвых" записей.
2. За время работы приложения некоторые данные уже могли устареть, но, тем не менее, хранятся в одной таблице с актуальными данными, из-за этого OLTP-транзакции начинают перемешиваться с OLAP-транзакциями, что приводит к долгим блокировкам и общим "тормозам" при обращении к данным.
3. Индексы на таблице значительно увеличиваются в объемах и становятся неэффективными, так как требуют выгрузки с диска значительного количества страниц индекса в произвольном порядке.

Одним из подходов для борьбы с объемными таблицами является **горизонтальное секционирование**.

Горизонтальное секционирование таблицы — физическое разделение одной таблицы на несколько подтаблиц, объединенных логически в одну таблицу, которая указывается непосредственно в DML-операторах SQL-запросов. Самым важным фактором является выбор критерия (ключа) секционирования, по которому происходит разделение данных. Критерием может являться как отдельная колонка, так и произвольное выражение. Помимо выбора ключа секционирования необходимо, также, выбрать метод секционирования.

Распространенные методы секционирования данных:

- по диапазону значений;
- по спискам значений.

В соответствии с методом, каждой секции задается набор маршрутизирующих допустимых значений, который, в дальнейшем, будет определять принадлежность строки данных к конкретной секции по значению ключа секционирования.

В результате образуется иерархическое дерево секций, состоящее из узлов — секционированных логических таблиц, определяющих критерий и метод секционирования и листьев — таблиц-секций, которые непосредственно хранят данные, подходящие под маршрутизирующее условие.

Преимущества, которые можно получить от горизонтального секционирования:

1. Дает возможность эффективно разделять OLTP- и OLAP-нагрузку, иначе говоря, отделить "горячие" данные от "холодных" в разных секциях. В результате уменьшается вероятность взаимоблокировок долгих и коротких транзакций, а также вытеснения "горячих" данных из кэша страниц.
2. Правильно выбранный ключ секционирования может значительно уменьшить объемы данных, выгружаемых с диска, за счет оптимального плана запроса, который сканирует только секции, подходящие под маршрутизирующее условие, а не всю таблицу целиком.
3. Autovacuum работает намного эффективнее, так как каждая секция физически является отдельной таблицей. Как следствие, можно эффективно удалять данные через удаление целых секций, в результате чего, не появляется "мертвых" записей и место освобождается моментально, без помощи autovacuum.
4. Поскольку индексы создаются по отдельности на каждой секции, их размер значительно меньше, относительно индекса на большой монолитной таблице. Как следствие, эффективность такого индекса выше, так как требуется меньше обращений к диску для считывания страниц индекса. Также есть возможность удалять или не создавать новых индексов на архивных данных, если они не требуются. За счет этого можно оптимизировать утилизацию дискового пространства.
5. В некоторых случаях можно отказаться от использования индексов совсем, так как, из-за небольших размеров, последовательное сканирование отдельных секций с последовательным доступом к диску может быть значительно эффективнее обращения к индексу, со случайным доступом к диску. При этом возрастает скорость вставки, обновления и удаления данных, так как не требуется обновлять ссылки в индексах.
6. Архивные секции могут быть перемещены в более дешевое и медленное физическое хранилище.

Обычно, принятие решения о секционировании таблицы зависит от конкретного приложения и данных, но, в общем случае, секционирование стоит рассмотреть как вариант при достижении таблицей объемов, сравнимых с объемами оперативной памяти сервера СУБД.

К недостаткам горизонтального секционирования можно отнести:

1. Неправильно выбранный критерий секционирования в лучшем случае не ухудшит производительность.
2. Смена ключа или метода потребует значительных накладных расходов по перестроению дерева секций.

3. Создание дерева секций и его расширение без поддержки со стороны СУБД может приводить к логическим ошибкам в "самодельной" реализации маршрутизации данных.
4. Глобальное ограничение уникальности, в т.ч. первичный ключ, трудно реализовать без поддержки со стороны СУБД.
5. Обновление колонок, входящих в ключ секционирования, может привести к необходимости переноса строки в другую секцию, что, опять же, требует поддержки со стороны СУБД.
6. Для обеспечения корректности данных, при включении секции в дерево, необходимо сделать полное сканирование секции на соответствие данных во всех записях маршрутизирующему ограничению.
7. Без поддержки со стороны СУБД и/или средств автоматизированного обновления схемы БД сложно поддерживать актуальный набор зависимых объектов секций таких как индексы, триггеры, внешние ключи и др.

Горизонтальное секционирование таблиц в PostgreSQL

В ходе эволюции PostgreSQL появилось два варианта горизонтального секционирования:

1. **Устаревший вариант** через наследование таблиц с маршрутизирующим триггером, начиная с 8 версии.
2. **Актуальный** на момент написания статьи вариант с декларативным секционированием, который появился начиная с 10 версии.

Оба варианта поддерживаются конвертером БД, поэтому рассмотрим по отдельности каждый из них для понимания общих принципов работы конвертера.

Секционирование через наследование таблиц с маршрутизирующим триггером ("триггерное секционирование")

Основой данного варианта является механизм наследования таблиц, с помощью которого можно создать базовую таблицу и наследовать от нее дочерние. Дочерние таблицы наследуют набор колонок и все DEFAULT-, CHECK- и NOT NULL-ограничения родительской таблицы. При обращении к родительской таблице SELECT-запросом данные выбираются из всех таблиц, входящих в иерархию наследования. То есть, благодаря наследованию, достигается логическая связь таблиц при доступе на чтение.

Для маршрутизации вставки данных в секции INSERT-запросами через корневую таблицу используется маршрутизирующий триггер. Он создается на корневой таблице на событие вставки, при этом, триггерная функция создается разработчиком самостоятельно. Триггерная функция инкапсулирует логику критерия секционирования, метода секционирования и маршрутизацию данных.

Чтобы защитить секции от ошибочных UPDATE- и INSERT-запросов, приводящих данные к несогласованному состоянию (неподходящему под маршрутизирующее ограничение), для каждой секции дополнительно создается CHECK-ограничение, которое реализует логику проверки данных в секции.

Общий алгоритм построения дерева "триггерного"-секционирования

1. Создать корневую таблицу, от которой будут наследоваться все секции.

```
1 CREATE TABLE master_table(  
2     id SERIAL NOT NULL,  
3     info TEXT  
4 );
```

2. Создать таблицы-секции, которые наследуются от корневой, при этом они не должны добавлять лишних колонок.

```
1 CREATE TABLE part_table1()  
2     INHERITS (master_table);  
3 CREATE TABLE part_table2()  
4     INHERITS (master_table);
```

3. Для каждой таблицы-секции создать CHECK-ограничение на соответствие данных условию маршрутизации для данной секции.

```
1 ALTER TABLE part_table1  
2     ADD CONSTRAINT part_table1_check CHECK (id < 100000);  
3 ALTER TABLE part_table2
```

```

4      ADD CONSTRAINT part_table2_check CHECK (id >= 100000 AND id <
      200000);

```

4. Для каждой таблицы-секции создать необходимые индексы, триггеры, ограничения.

```

1  ALTER TABLE part_table1
2      ADD CONSTRAINT part_table1_pk PRIMARY KEY (id);
3  ALTER TABLE part_table2
4      ADD CONSTRAINT part_table2_pk PRIMARY KEY (id);

```

5. Создать триггер на вставку в корневую таблицу, который будет маршрутизировать новые записи.

```

1  CREATE OR REPLACE FUNCTION master_ins_part_route()
2      RETURNS trigger
3      LANGUAGE 'plpgsql'
4      COST 100
5      VOLATILE
6  AS
7  $BODY$
8  BEGIN
9      IF NEW.id < 100000 THEN
10         INSERT INTO part_table1 VALUES (NEW.*);
11     ELSIF NEW.id < 200000 THEN
12         INSERT INTO part_table2 VALUES (NEW.*);
13     ELSE
14         RAISE 'No route for partition key value %', NEW.id;
15     END IF;
16     RETURN NULL;
17 END
18 $BODY$;
19 CREATE TRIGGER master_ins_part_trig
20     BEFORE INSERT
21     ON master_table
22     FOR EACH ROW
23     EXECUTE PROCEDURE master_ins_part_route();

```

6. Убедиться, что выставлен параметр конфигурации `constraint_exclusion`, так как, без него, планы запросов будут неоптимальными.

Ограничения "триггерного" варианта

1. Корректность данных в секции зависит от CHECK-ограничения, создаваемого разработчиком.
2. Операторы INSERT и COPY не вставляют записи автоматически в нужную секцию, а зависят от маршрутизирующего триггера, что замедляет вставку данных из-за постоянного вызова триггерной функции.
3. Маршрутизирующий триггер создается разработчиком, поэтому может содержать логические ошибки.
4. Ограничение уровня вложений — увеличение количества маршрутизирующих узлов потребует усложнения CHECK-ограничений и логики триггерной функции.
5. Ограничение добавления новой секции в дерево — необходимо обновлять триггерную функцию самостоятельно при каждом добавлении новой секции в дерево.
6. Невозможно создание глобальных ограничений уникальности, в т.ч. ограничений первичного ключа.
7. Актуальную структуру зависимых объектов на секциях (таких как триггеры, индексы, ограничения) необходимо поддерживать самостоятельно.
8. Секционированные таблицы и таблицы секции неразличимы, эти роли определяются триггерной функцией.

Декларативное секционирование таблиц (PostgreSQL 10+)

Данный вид секционирования основан на расширенном наборе ключевых слов SQL, которые можно указывать в CREATE/ALTER-скрипте таблицы, что позволяет в декларативном виде строить дерево секций с помощью DDL-запросов. Вся логика по маршрутизации вставки/обновления и проверке согласованности данных, при этом, инкапсулируется непосредственно ядром СУБД.

Декларативное секционирование разделяет понятия секционированной таблицы и таблицы секции:

- Секционированная таблица (partitioned table) — таблица, которая определяет критерий и метод секционирования, при этом она не хранит данные и служит только для маршрутизации вставки в секции, по сути является логической таблицей. Секционированные таблицы могут быть включены как секции другой секционированной таблицы, таким образом, возможно строить дерево секционирования произвольной вложенности.
- Таблица-секция (partition table) — таблица, которая подключается к секционированной таблице с указанием маршрутизирующей информации, соответствующей требованиям метода секционирования, заданного на секционированной таблице. Является непосредственным хранилищем данных.

Декларативное секционирование, также, использует механизм наследования в своей реализации, но при этом накладывает два дополнительных ограничения:

1. Формат таблицы секции не может содержать дополнительных полей.
2. От секционированных таблиц нельзя наследоваться явно.

Как следствие, использования механизма наследования таблиц, при декларативном секционировании все CHECK-, DEFAULT- и NOT NULL-ограничения, на секционированной таблице, автоматически наследуются. Унаследованное ограничение можно удалить только на родительской секционированной таблице, но не на самой секции.

Декларативные секции можно создавать как непосредственно операторами в CREATE-скрипте, так и конвертировать существующие таблицы в таблицы-секции с помощью инструкции ALTER TABLE ATTACH PARTITION. Возможен обратный сценарий, когда секцию нужно исключить из дерева секционирования используется DDL-инструкция ALTER TABLE DETACH PARTITION.

Декларативное секционирование поддерживает следующие методы секционирования:

1. **RANGE** — секционирование таблицы по набору диапазонов. Ключ секционирования может состоять как из одной, так и из нескольких колонок. Ядро СУБД проверяет пересечения заданных диапазонов.

```
1 CREATE TABLE by_range_base(  
2     i INT,  
3     j INT  
4 ) PARTITION BY RANGE(mod(i,10),div(j,2));  
5 CREATE TABLE by_range_part  
6     PARTITION OF by_range_base  
7     FOR VALUES  
8     -- FROM ( { numeric_literal | string_literal | TRUE | FALSE |  
MINVALUE | MAXVALUE } [, ...] )  
9     -- TO ( { numeric_literal | string_literal | TRUE | FALSE |  
MINVALUE | MAXVALUE } [, ...] )  
10    FROM (1, MINVALUE) TO (MAXVALUE, MAXVALUE);
```

2. **LIST** — секционирование таблицы по спискам значений. Ключ секционирования состоит только из одной колонки, значения которой входят в списки маршрутизации. Ядро СУБД проверяет пересечения списков значений.

```
1 CREATE TABLE by_list_base(  
2     i INT  
3 ) PARTITION BY list(mod(i,10));  
4 CREATE TABLE by_list_part  
5     PARTITION OF by_list_base  
6     -- { numeric_literal | string_literal | TRUE | FALSE | NULL } [,  
...]  
7     FOR VALUES IN (1,2,3);
```

3. **HASH** (Postgres 11+) — секционирование таблицы по вычислению остатка от деления по модулю. Ключ секционирования состоит только из одной колонки, значение которого делится по модулю. Ядро СУБД проверяет пересечения значений, а также неравенство REMINDER < MODULUS. Маршрутизация осуществляется по формуле: <ключ секционирования> % <MODULUS> == <REMAINDER>.

```
1 CREATE TABLE by_hash_base (  
2     i INT  
3 ) PARTITION BY HASH (i);  
4 CREATE TABLE by_hash_part0 PARTITION OF by_hash_base  
5     FOR VALUES WITH (MODULUS 3, REMAINDER 0);  
6 CREATE TABLE by_hash_part1 PARTITION OF by_hash_base  
7     FOR VALUES WITH (MODULUS 3, REMAINDER 1);
```

```

8 CREATE TABLE by_hash_part2 PARTITION OF by_hash_base
9     FOR VALUES WITH (MODULUS 3, REMAINDER 2);

```

Общий алгоритм построения декларативного дерева секционирования

1. Создать корневую секционированную таблицу.

```

1 CREATE TABLE master_table(
2     i INT
3 ) PARTITION BY RANGE(i);

```

2. Опционально создать вложенные узлы секционирования.

```

1 CREATE TABLE dir_table1
2     PARTITION OF master_table
3     FOR VALUES FROM (100000) TO (200000)
4     PARTITION BY LIST(i);
5 CREATE TABLE dir_table2
6     PARTITION OF master_table
7     FOR VALUES FROM (200000) TO (300000)
8     PARTITION BY HASH(i); -- pg 11 feature

```

3. Создать таблицы-секции в узлах секционирования.

```

1 CREATE TABLE master_part_table1
2     PARTITION OF master_table
3     FOR VALUES FROM (0) TO (100000);
4 CREATE TABLE dir_part_table1
5     PARTITION OF dir_table1
6     FOR VALUES IN (100000, 100002, 100003);
7 CREATE TABLE dir_part_table2
8     PARTITION OF dir_table2
9     FOR VALUES WITH (MODULUS 10, REMAINDER 5); -- pg 11 feature

```

4. Опционально создать секции по умолчанию (Postgres 11+).

```

1 CREATE TABLE master_default_part
2     PARTITION OF master_table
3     DEFAULT;
4 CREATE TABLE dir1_default_part
5     PARTITION OF dir_table1
6     DEFAULT;
7 -- there is no default partition available for HASH-partitioned table

```

5. Опционально добавить секции из обычных таблиц.

```

1 CREATE TABLE ordinal_table
2     (LIKE master_table);
3 ALTER TABLE master_table
4     ATTACH PARTITION ordinal_table
5     FOR VALUES FROM (300000) TO (400000);

```

6. Опционально создать необходимые индексы, триггеры, ограничения для каждой секции.

```

1 ALTER TABLE master_part_table1
2     ADD CONSTRAINT master_part_table1_pk PRIMARY KEY (i);
3 ALTER TABLE master_default_part
4     ADD CONSTRAINT master_default_part_pk PRIMARY KEY (i);
5 ALTER TABLE dir_part_table1
6     ADD CONSTRAINT dir_part_table1_pk PRIMARY KEY (i);
7 ALTER TABLE dir1_default_part
8     ADD CONSTRAINT dir1_default_part_pk PRIMARY KEY (i);
9 ALTER TABLE dir_part_table2

```

```

10     ADD CONSTRAINT dir_part_table2_pk PRIMARY KEY (i);
11 ALTER TABLE ordinal_table
12     ADD CONSTRAINT ordinal_table_pk PRIMARY KEY (i);

```

Ограничения "декларативного" варианта в PostgreSQL версии 10

Поскольку декларативное секционирование — свежая фиша, которая появилась в 10 версии, присутствует ряд ограничений, которые стоит учитывать при разработке приложения:

1. Невозможно конвертировать обычную таблицу в секционированную и обратно без пересоздания DROP + CREATE.
2. Невозможно создавать глобальные ограничения уникальности.
3. Полное сканирование CHECK-ограничением при подключении секции.
4. Изменение ключа или метода секционирования приведет к "пересборке" всего поддерева секций инструкциями DETACH + ATTACH.
5. Актуальную структуру зависимых объектов на секциях (таких как триггеры, индексы, ограничения) необходимо поддерживать самостоятельно.
6. На секционированные таблицы (в т.ч. корень секционирования) нельзя ссылаться ограничениями внешнего ключа.
7. Отсутствуют таблицы-секции по умолчанию. Если строку данных не удалось маршрутизировать в секцию, то будет ошибка.
8. Обновление строки, приводящее к изменению значения ключа секционирования до выхода за пределы допустимых для секции значений, не перенесет строку в другую секцию, вместо этого UPDATE завершится с ошибкой.

Доработки "декларативного" варианта в PostgreSQL версиях 11 и 12

Поскольку все крупные возможности в PostgreSQL внедряются по частям, то в последующих версиях можно ожидать значительные улучшения декларативного секционирования.

С выходом 11 версии:

1. Появился новый метод секционирования по **HASH**-значениям ключа секционирования.
2. UPDATE-оператор может перенести строку в другую секцию, если требуется.
3. Появились секции с маршрутом по умолчанию, если для записи не нашлось подходящей секции, но только для методов секционирования **LIST** и **RANGE**. На одной секционированной таблице возможно создание только одной секции по умолчанию.
4. Разрешен оператор INSERT ON CONFLICT UPDATE для секционированных таблиц.
5. Добавлена поддержка DDL-инструкций для секционированных таблиц: CREATE INDEX, UNIQUE, PRIMARY KEY, TRIGGER FOR EACH ROW, FOREIGN KEY. При этом глобальное ограничение уникальности (UNIQUE, PRIMARY KEY) достижимо, только если колонка является ключом секционирования. На секционированные таблицы все еще нельзя ссылаться внешним ключом.

С выходом 12 версии:

1. Оптимизирована работа для случаев с большими количествами секций в дереве.
2. Внешние ключи могут ссылаться на секционированные таблицы.
3. В ключ секционирования для метода **RANGE** можно включать выражения.
4. Добавлена функция для визуализации дерева секционирования pg_partition_tree.

Поддержка секционирования в платформенном конвертере БД

Конвертером БД для СУБД PostgreSQL поддерживаются оба варианта секционирования: "триггерный" и "декларативный". Дерево секционирования для таблицы настраивается редактированием свойства "Partition directories" в описании таблицы в словаре [dicx](#).

Свойство "Partition directories" представляет собой плоский список директорий секционирования, объединенных в иерархию с помощью пути до директории.

^ Partition directories 		
Directory path	Partition description	Directory partitions
/	Range partitioned by (i)	[p1,p2]
/d1	Range partitioned by (i) for values in range (1) to (200)	[p3]
/d1/d3	Hash partitioned by (i) for values in range (1) to (300)	No partitions specified
/d2	List partitioned by (i) for values in range (1) to (250)	[p6]

Директория секционирования — абстракция, позволяющая описать структуру секционированной таблицы.

Список должен обязательно начинаться с корневой директории с путем `/`, так как корневая директория — точка входа для работы с деревом секционирования. Все последующие директории наследуются от корневой директории.

Полноценная поддержка вложенных деревьев секционирования реализована только для декларативных методов секционирования. Для "триггерного" варианта все ограничивается одной корневой директорией, ввиду сложности и неэффективности реализации нескольких уровней вложенности с помощью триггерной функции.

Partition tree directory

Directory name

d3

Directory parent

/d1

Routing left range bounds

1

Routing right range bounds

120

Partitioning type

Hash partitioned

Partitioning key expression

i

Dynamic partitions

☐

Object filters

Filter type	Filter object	Partition name pattern	Object name pattern
Block all matching	Index	p.*	.*i1
Block all matching	Unique constraint	*	*

Directory partitions

Partition name	Is default
p4	☒
p5	☐

Путь до директории имеет вид, схожий с путем до директории в файловой системе и необходим для образования логической иерархии директорий. Он состоит из двух базовых компонент:

- Уникальное имя директории "Directory name" для связи с поддиректориями.
- Путь до родительской директории "Directory parent", который позволяет определить позицию текущей директории в иерархии дерева секционирования.

Все имена в списке (директорий и таблиц-секций) должны быть уникальными на всех уровнях, то есть не допустимо наличие директории `/a/b/c` и таблицы-секции с именем `c` в директории `/d/e/f`, также запрещены варианты с одинаковыми именами директорий на разных уровнях, например, `/a/b/c` и `/d/e/f/a`. Данное ограничение введено, так как в БД дерево разворачивается в плоский список таблиц, связанных между собой, с ограничением на максимальную длину идентификатора в 64 байта. При этом должно соблюдаться ограничение уникальности имени каждой из таблиц, входящих в дерево секционирования.

Далее следует переменный набор свойств маршрутизации. Конкретное количество и названия свойств зависит от метода секционирования родительской директории и определяют критерий маршрутизации данных с верхнего уровня в текущий. Для корневой директории эти свойства отсутствуют, так как в них нет смысла. Значения из этих полей подставляются в `FOR VALUES <ROUTING_EXPR>`, либо в генерируемую конвертером триггерную функцию, в случае секционирования по триггеру.

Возможные варианты наборов маршрутизирующих свойств:

- Если родительская директория секционирована по методу **RANGE**, то это будут свойства "Routing left range bounds" и "Routing right range bounds". В них, через запятую, указываются левые и правые границы поддиапазонов значений, входящих в ключ секционирования, без указания предлогов `FROM` и `TO` и круглых скобок.

Routing left range bounds

5000,1,10

Routing right range bounds

10000,10,20

- В случае если родительская директория секционирована по методу **LIST**, то это будет свойство "Routing values list", где через запятую необходимо указать список значений ключа секционирования, без указания предлога `IN` и круглых скобок.

Routing values list

1,2,3,4,5

- Если родительская директория секционирована по методу **HASH**, то это будут свойства "Routing hash modulus" и "Routing hash remainder". В них указываются два целых значения — делитель по модулю и маршрутизирующий остаток от деления, без указания служебных слов MODULUS и REMAINDER и круглых скобок.

Routing hash modulus ⓘ	10
Routing hash remainder ⓘ	5

- Если родительская директория секционирована триггером, то это будет свойство "Routing trigger condition". В нем указывается произвольное выражение, которое должно возвращать true для подходящих данной секции записей. Само значение подставляется в генерируемую конвертером маршрутизирующую триггерную функцию, в оператор IF <routing_trigger_condition1> THEN ... ELSIF <routing_trigger_condition2> THEN ... ELSE ... END.

Routing trigger condition ⓘ	<code>i % 10 = 5</code>
-----------------------------	-------------------------

В свойстве "Partitioning type" выбирается метод секционирования для текущей директории. Значение из этого поля подставляется в PARTITION BY <METHOD>....

Доступны следующие варианты:

- Trigger partitioned** — "триггерное" не декларативное секционирование, является устаревшим и оставлено для поддержки обратной совместимости. Основные ограничения: * данный вид доступен только для корневой директории (не может быть выбран для декларативной поддиректории по очевидным причинам); * для корневой директории с данным типом нельзя добавлять поддиректории (вложенность не поддерживается из-за неэффективности реализации триггерной функции).
- Range partitioned** — декларативное секционирование по диапазонам (**RANGE**).
- List partitioned** — декларативное секционирование по спискам значений (**LIST**).
- Hash partitioned** — декларативное секционирование по остатку от деления по модулю (**HASH**).

В свойстве "Partitioning key expression" задается ключ секционирования для текущей директории. Им может быть имя колонки или выражение, в зависимости от выбранного метода секционирования. Значение из этого поля подставляется в PARTITION BY <METHOD>(<PART_KEY>).

Динамическое секционирование (свойство "Dynamic partitions")

Очень часто при разработке БЛ количество секций на таблице заранее не определено. Поэтому возникает задача создания секций "на лету", прямо во время работы приложения. Данная задача осложняется следующими факторами:

- "Из коробки" PostgreSQL (на момент написания статьи — 12 версия) не поддерживает динамическое секционирование. Создание таблицы секции можно реализовать через триггер, но поддержание актуального набора зависимых объектов, таких как индексы, остается проблемой.
- Конвертер БД анализирует состояние схемы и удаляет все, что не описано в проекте.

Для преодоления этих сложностей в директории секционирования было введено свойство "Dynamic partitions". При выставлении данного свойства, конвертер начинает воспринимать секции директории так, как если бы они были описаны в проекте и поддерживать на каждой из секций актуальное состояние зависимых объектов. При этом, в таких директориях допустимо создавать набор predetermined секций.

Также в платформенный сервисный модуль "DB Converter Postgres" добавлено два private-метода "DBConverter.CreatePartition" для создания одной секции и "DBConverter.CreatePartitions" для создания множества динамических секций с помощью БЛ приложения. При вызове данных функций будет создано заданное количество динамических секций, в заданной директории, с актуальным набором зависимых объектов, которые были описаны в проекте при последней конвертации. Примеры вызова можно найти в описании функций в справочник .

Ограничения динамического секционирования:

- поддерживаются только динамические секции на уровне директорий, работа с динамически созданными директориями не поддерживается;
- свойство "Dynamic partitions" не наследуется поддиректориями, для каждой директории выставляется отдельно;
- так как конвертер БД не может удалять секции из директории с динамическими секциями, разработчик обязуется разработать самостоятельную логику по удалению устаревших секций;
- вызов БЛ методов по созданию динамических секций возможен только для директорий, у которых выставлен флаг динамического секционирования, иначе будет ошибка.

Выборочная фильтрация объектов (свойство "Object filters")

Иногда, при разработке схемы секционирования данных, не все секции эквивалентны. Из-за этого возникает необходимость фильтрации зависимых объектов на конкретных секциях.

Например:

- секция стала архивной, и для оптимизации дискового пространства для нее необходимо удалить все индексы или не создавать новых;
- секция разрослась в объемах, и, в связи с этим, требуется создать вспомогательный индекс только на ней.

Для решения данной проблемы был разработан интерфейс создания списка фильтрующих правил на уровне директорий.



Partition tree directory filter

Filter type	Block all matching
Filter object	Index
Partition name pattern	p.*
Object name pattern	.*i1

Интерфейс фильтрующего правила состоит из следующих свойств:

- "Filter type" определяет тип фильтра:
 - **Block all matching** — заблокировать все, что подходит под условие (черный список).
 - **Allow only matching** — разрешать только то, что подходит под условие (белый список).
- "Filter object" — задает класс объектов схемы БД, к которым применимо данное правило.
- "Partition name pattern" — имя секции, для которой применимо данное правило (не обязательно только для секций из текущей директории, возможна фильтрация и связанных секций из поддиректорий). Может содержать как имя конкретной секции, так и REGEX-шаблон целой группы секций, а также специальное значение *, обозначающее "все секции". В поле указывается чистое имя секции без префикса родительской таблицы (см. раздел про структуру таблиц).
- "Object name pattern" — имя объекта в схеме БД, для которого применимо данное правило. Может содержать как имя конкретного объекта, так и REGEX-шаблон целой группы объектов, а также специальное значение *, обозначающее "все объекты на секциях с заданным типом". В поле указывается полное имя/шаблон под полное имя объекта в БД.

При создании динамических секций с помощью БЛ методов фильтры учитываются.

Правила можно комбинировать на разных уровнях для достижения точной фильтрации по секциям и объектам.

Правила фильтрации обходятся в обратном порядке: от конца к началу списка в текущей директории, от текущей директории до корневой с учетом иерархии вложенности.

Решение о фильтрации объекта принимается по результату первого правила фильтрации в цепочке, параметры которого будут подходить под состояние объекта.

Например, если требуется создать индекс только на одной секции, то необходимо создать правило в корневой директории, которое заблокирует создание индекса на всех секциях. После него добавить правило в директорию с секцией, которое разрешит создание индекса на требуемой секции.

Пример 1. Блокирующий фильтр индекса i1 на корневой директории для всех секций



Partition tree directory filter

Filter type	Block all matching
Filter object	Index
Partition name pattern	.
Object name pattern	.*i1

Пример 2. Белый фильтр, разрешающий индекс для секции p5, ранее заблокированный в корне

Partition tree directory filter

Filter type ⓘ

Allow only matching

Filter object ⓘ

Index

Partition name pattern ⓘ

p5

Object name pattern ⓘ

.*i1

Список таблиц-секций в директории (свойство "Directory partitions")

В каждой директории можно описать список, заранее определенных, секций.

Partition tree directory entry

Partition name ⓘ

p3

Is default partition ⓘ

☐

Routing left range bounds ⓘ

3

Routing right range bounds ⓘ

50

Интерфейс создания новой секции состоит из следующих свойств:

- "Partition name" — уникальное имя секции, не должно совпадать ни с одним именем директории или секции в текущем дереве секционирования (см. структуру таблиц).
- "Is default partition" — признак секции по умолчанию (данная возможность присутствует только в версии PostgreSQL 11+). В каждой директории можно создать не более одной секции по умолчанию.
- Набор маршрутизирующих свойств — зависит от метода секционирования в родительской директории, функциональное назначение полностью совпадает с аналогичными свойствами в интерфейсе редактирования маршрутизации для директории.

Структура таблиц дерева секционирования

Несмотря на то, что при редактировании в [Genie](#) структура дерева секционирования представлена в иерархическом виде, на уровне базы данных, данное дерево разворачивается в плоский список таблиц. Решение обусловлено несколькими факторами:

1. С плоским списком проще работать при анализе состояния схемы.
2. Все связи между таблицами сохраняются на уровне ядра СУБД за счет наследования, избыточность не требуется.
3. Имя идентификатора ограничено 64 байтами, все что больше — просто обрезается. Поэтому нет возможности надежно инкапсулировать полный путь до директории или секции в имени таблицы.

Например, есть таблица "PartTable", для которой в метаданных описано следующее дерево секций:

```

1 / RANGE [a, b, c]
2 +---/d1 LIST [d, e, f]
3     +---/d2 HASH [g, h, j]
```

На уровне базы данных дерево секционирования будет представлено в виде списка из 12 связанных таблиц:

```

1 -- корень секционирования /
2 CREATE "PartTable" (...)
3     PARTITION BY RANGE (...);
4
5 -- директория /d1
6 CREATE "PartTable.$d1"
7     PARTITION OF "PartTable"
8     FOR VALUES FROM (...) TO (...)
9     PARTITION BY LIST(...);
```

```

10
11 -- директория /d1/d2
12 CREATE "PartTable.$d2"
13     PARTITION OF "PartTable.$d1"
14     FOR VALUES IN (...);
15
16 -- секции корневой директории
17 CREATE "PartTable.$a"
18     PARTITION OF "PartTable"
19     FOR VALUES FROM (...) TO (...)
20 CREATE "PartTable.$b"
21     PARTITION OF "PartTable"
22     FOR VALUES FROM (...) TO (...)
23 CREATE "PartTable.$c"
24     PARTITION OF "PartTable"
25     FOR VALUES FROM (...) TO (...)
26
27 -- секции директории /d1
28 CREATE "PartTable.$d"
29     PARTITION OF "PartTable.$d1"
30     FOR VALUES IN (...);
31 CREATE "PartTable.$e"
32     PARTITION OF "PartTable.$d1"
33     FOR VALUES IN (...);
34 CREATE "PartTable.$f"
35     PARTITION OF "PartTable.$d1"
36     FOR VALUES IN (...);
37
38 -- секции директории /d1/d2
39 CREATE "PartTable.$g"
40     PARTITION OF "PartTable.$d2"
41     FOR VALUES WITH (MODULUS ..., REMAINDER ...);
42 CREATE "PartTable.$h"
43     PARTITION OF "PartTable.$d2"
44     FOR VALUES WITH (MODULUS ..., REMAINDER ...);
45 CREATE "PartTable.$j"
46     PARTITION OF "PartTable.$d2"
47     FOR VALUES WITH (MODULUS ..., REMAINDER ...);

```

Не трудно заметить, что имена таблиц, входящих в дерево секционирования, состоят из имени корневой секционированной таблицы, служебного разделителя .\$ и имени сущности в дереве секционирования. Поскольку имя таблицы должно быть уникальным, то и все имена в дереве секций должны быть уникальными.

Что происходит при изменениях в описании дерева секционирования?

Важно понимать, что **не существует** ALTER-инструкций для изменения только ключа и метода секционирования, а также маршрутизирующего выражения и родительской секционированной таблицы. Конвертер может только "пересобирать" дерево секционирования с переподключением секций с помощью инструкций ALTER TABLE ... DETACH PARTITION ... и ALTER TABLE ... ATTACH PARTITION ... FOR VALUES Операция ATTACH **не бесплатная**, так как при подключении происходит неизбежное полное сканирование секции, потому что ядро СУБД прогоняет внутреннее, автоматически сгенерированное, CHECK-ограничение для всех записей.

Примеры типовых действий по изменению и их последствий:

- Изменилось имя секции/директории:
 1. Необходимо создать декларативное расширение по переименованию секции таблицы.

2. При конвертации секция/директория будет переименована инструкцией ALTER TABLE ... RENAME ... с использованием промежуточного имени, чтобы не возникало конфликтов.
- Изменилось маршрутизирующее выражение у таблицы секции/директории:
 1. Секция/директория отключена от дерева DETACH-инструкцией.
 2. Секция/директория подключена заново к своему родителю с новым маршрутизирующим выражением в ATTACH-инструкции.
 - Изменился родитель у секции/директории:
 1. Секция/директория отключена от дерева DETACH-инструкцией.
 2. Секция/директория подключена к новому родителю с помощью ATTACH-инструкции.
 - Изменился метод секционирования таблицы:
 1. Все секции от этой таблицы будут отключены DETACH-инструкцией.
 2. Сама секционированная таблица, также, будет отключена от своей родительской таблицы, если такая имеется, DETACH-инструкцией.
 3. Секционированная таблица пересоздается с новым методом секционирования с помощью последовательности инструкций DROP и CREATE.
 4. Секционированная таблица подключается к своей родительской таблице, если такая имеется, с помощью ATTACH-инструкции.
 5. Все секции таблицы подключаются обратно ATTACH-инструкциями.
 - Изменился ключ секционирования — все действия идентичны предыдущему пункту.
 - Изменился вариант секционирования с "триггерного" на "декларативный" или наоборот (данное преобразование возможно только для деревьев секционирования без вложенных поддиректорий):
 1. Все таблицы-секции отключаются от дерева DETACH-инструкциями, при переходе на "триггерное" секционирование и NO INHERIT-инструкциями при переходе на декларативное секционирование.
 2. Корень секционирования пересоздается с помощью последовательности инструкций DROP и CREATE.
 3. Все таблицы-секции подключаются обратно с помощью ATTACH-, либо INHERIT-инструкций.
 4. Если был переход на "триггерный" вариант, то конвертером БД создается автогенерируемый маршрутизирующий триггер на вставку.

Для остальных изменений, таких как:

- подключение сторонней таблицы в состав дерева секционирования;
- отключение секции из состава дерева в отдельную таблицу;
- смена типа таблицы с "узла" на "лист" и обратно.

потребуется создание расширений конвертации, либо скриптов, так как в этом случае требуется нетривиальное перемещение данных и корректная обработка метаданных первичного ключа.